HTML-CSS-01 第1回_まとめ

第1回の目標

- **ブログサイトのトップページを作ろう**
- **HTMLとCSSの記述に慣れよう**
- **コーディングを楽しもう**

ブログサイトとは

目的

自分の出来事やそれに対する意見、あるいはWebサイトに対する情報を「記事」という形で公開しているサイト。

基本構成

- 全ての記事一覧
- 記事の本体
- 分類ごとの記事一覧

役割	使用するhtmlファイル
全ての記事一覧	index.html
記事の本体	single.html
分類ごとの記事一覧	category.html

モバイルファースト

スマートフォンからのアクセス増加に伴い、スマートフォンでの使いやすさを優先して、デザイン・コーディングを行う考え方

HTML要素の表示形式

変更する場合はCSSの`display`プロパティを使用

block

- 要素が縦方向に並ぶ表示形式
- 1つの要素が1行分の幅を使う→次の要素は必ず下の行にくる
- 幅や高さ余白などの調整が可能
    - `margin`, `padding`が使える
- 見出し・段落・セクションなど、ページ構成の土台となる要素に使われる。

inline

- 要素が横方向に並ぶ表示形式
- テキストと同じ行の流れの中に配置されるため、文章の途中でも自然に表示される
- 幅や高さは指定できない
    - 上下方向のmarginも基本的に使えない。
    - paddingは指定できるが、行中の要素であるため、上下方向に指定するとレイアウトが崩れる原因になる
- リンクや文字装飾など、テキストの一部を装飾・操作したい場面で使われる。

inline-block

- blockとinlineの両方の性質をあわせ持つ表示形式- 要素はinlineと同じく横方向に並ぶ
- block と同じく様々な指定が可能
    - width（幅）
    - height（高さ）
    - margin
    - padding

各HTMLタグの表示形式の初期値

HTMLタグ	表示形式の初期値
div	block
h1	block
section	block
article	block
header	block

HTMLタグ	表示形式の初期値
main	block
footer	block
a	inline
span	inline
img	inline
iframe	inline
video	inline

flex box

Flexアイテムの性質

- 横並びで表示される
- 親要素に合わせて横幅・高さが伸縮する
- HTMLの記述が表示順に影響しない

親要素にdisplay: flex;を指定すると、子要素が横並びになる

```
<ul class="flexbox">
 <li class="flex-item">要素１</li>
 <li class="flex-item">要素２</li>
 <li class="flex-item">要素３</li>
</ul>
```

```
.flexbox {
 display: flex;
}

.flex-item {
 list-style: none;
 background: #ffbb1e;
 color: #fff;
 padding: 10px;
```

```
 margin: 10px;
}
```

- 親要素:flexコンテナ という
- 子要素:flexアイテム という

flexコンテナ（親要素）に指定するプロパティ

```
<div class="flexbox row">
    <div class="flex-item row01">①</div>
    <div class="flex-item row02">②</div>
    <div class="flex-item row03">③</div>
  </div>
```

```
.flexbox {
  text-align: center;
  display: flex;
  margin: 10px 0;
}
.row{
  flex-direction: row;
}
.flex-item{
  color: #fff;
  width: 30%;
}
.row01{
  background-color: red;
}
```

プロパティ	指定する対象	値	機能
flex-direction	並び方	row（初期値）	横並び 左寄せ
		row-reverse	横並び 右寄せ
		column	縦並び 上寄せ
		column-reverse	縦並び 下寄せ

プロパティ	指定する対象	値	機能
flex-wrap	折り返し	wrap	親要素からはみ出た部分を折り返す
		nowrap	折り返さない
		wrap-reverse	逆順で折り返す
frex-flow	並び方と折り返し	上2つを合わせたもの	書式：flex-flow: (flex-directionの値) (flex-wrapの値);
justify-content	横の配置	frex-start	左寄せ
		frex-end	右寄せ
		center	中央寄せ
		space-between	両端揃え
		space-arownd	それぞれの両端に均等な余白をつけて配置
		space-evenry	両端を含めた全ての余白を均等に配置
align-items	高さ	stretch/normal（初期値）	親要素の高さに合わせて子要素を伸縮する
		frex-start	親要素の上端で揃える
		flex-end	親要素の下端で揃える
		center	親要素の中央で揃える
		baseline	子要素をベースラインで揃える
align-content	縦の配置	flex-start	上寄せ
		flex-end	下寄せ
		center	中央寄せ
		space-between	上下の両端揃え
		space-arownd	それぞれの上下の両端に均等な余白をつけて配置
		space-evenry	上下の両端を含めた全ての余白を均等に配置

flexアイテム（子要素）に指定するプロパティ

```
<div class="flex-box">
  <div class="flex-item item01">①</div>
  <div class="flex-item item02">②</div>
```

```
  <div class="flex-item item03">③</div>
</div>
```

```
.flex-box{
  display: flex;/*子要素をflexアイテムにする*/
  border: 1px solid #000;
}

.flex-item{
  /*見た目のスタイル*/
  border: 1px solid #000;
  height: 100px;
  font-size: 24px;
  color: #fff;

  /*テキストを上下左右中央寄せにするスタイル*/
  display: flex;
  align-items: center;/*上下の中央寄せ*/
  justify-content: center;/*左右の中央寄せ*/
}

.item01{
  flex-grow: 1;
  background-color: red;
}
```

プロパティ	役割	備考
order	表示順を指定	数値が小さいものから表示される（初期値：0）
flex-grow	横幅の調整	flexコンテナ内に余白があるとき、余白部分をflexアイテムに指定した比率の分だけ追加 初期値：0
flex-shrink	縮み率を指定	指定の子要素が他の子要素と比較してどれくらい縮むか指定する 子要素は親要素の幅をはみ出したときに自動的に縮小し、均等な大きさになる性質がある 初期値：1

プロパティ	役割	備考
flex-basis	横幅もしくは高さを指定	auto（初期値）：flexアイテムの横幅を自動的に広げる 数値 + 単位 ：flexアイテムの横幅を、数値とpxやrem、emなどの単位で指定する 数値 + % ：flexアイテムの横幅を%比率で指定する 併用した場合にはflex-basisが優先される
align-self	個々の縦の配置を指定	auto（初期値）：親要素のalign-itemsの値に合わせて子要素を配置する flex-start：親要素の上端で揃える flex-end：親要素の下端で揃える center：親要素の中央で揃える baseline：子要素をベースラインで揃える stretch：親要素に合わせて子要素を伸縮する）
gap	余白	row-gap（縦方向の余白）とcolumn-gap（横方向の余白） gap 32px,80px; なら row-gap=32px,colum-gap=80px gap 32px; なら row-gap=32px,colum-gap=32px

flexショートバンド

指定のタイプ	単位	設定されるプロパティ	記述例	展開形（実際の指定）
1項目	なし	`flex-grow`	`flex: 2;`	`flex: 2 1 0%;`
	あり	`flex-basis`	`flex: 200px;`	`flex: 1 1 200px;`
2項目	なし	`flex-grow`, `flex-shrink`	`flex: 2 1;`	`flex: 2 1 0%;`
	あり（2つ目）	`flex-grow`, `flex-basis`	`flex: 2 200px;`	`flex: 2 1 200px;`
3項目	あり（3つ目）	`flex-grow`, `flex-shrink`, `flex-basis`	`flex: 2 1 200px;`	`flex: 2 1 200px;`
キーワード	-	全プロパティ（一括設定）	`flex: initial;`	`flex: 0 1 auto;`
			`flex: auto;`	`flex: 1 1 auto;`
			`flex: none;`	`flex: 0 0 auto;`

疑似クラス

HTML側にクラス（class="..."）を追加することなく、要素の「特定の状態や条件」に応じてスタイルを適用するCSSの仕組み。 - 操作・状態の指定：マウスが乗った（:hover）、選択中（:focus）などのユーザー操作 - 位置・条件の指定：最初の子要素（:first-child）、偶数番目（:nth-child(even)）などの位置指定

書式

```
セレクタ:擬似クラス名(引数) {
  プロパティ: 値;
}
```

例

```
<ul>
  <li>1行目</li>
  <li>2行目（色が変わる）</li>
  <li>3行目</li>
  <li>4行目（色が変わる）</li>
</ul>
```

```
/* 偶数（2, 4, 6...）番目のliだけ背景色を変更 */
li:nth-child(even) {
  background-color: #f2f2f2;
}
```

よく使用する疑似クラス

- `:first-child`：親要素内の「最初の子要素」を指定
  - 基本指定：同じ親要素の中で一番最初にある要素に対してスタイルを当てる
    - 例：`li:first-child { color: red; }`
- `:last-child`：親要素内の「最後の子要素」を指定
  - 基本指定：同じ親要素の中で一番最後にある要素に対してスタイルを当てる
    - 例：`li:last-child { margin-bottom: 0; }`
- `:nth-child(n)`：親要素内の「n番目の子要素」を指定
  - 数値指定：指定した順番（数値）の子要素にスタイルを当てる
    - 例：`li:nth-child(3) { font-weight: bold; }`
  - 奇数・偶数指定：`odd`（奇数）や `even`（偶数）でまとめて指定する
    - 例：`li:nth-child(even) { background-color: #f0f0f0; }`
  - 倍数指定：`2n`（2の倍数）や `3n`（3の倍数）などで周期的に指定する
    - 例：`li:nth-child(2n) { color: red; }`
- `:nth-of-type(n)`：指定した要素の種類の中で「n番目の要素」を指定
  - 同一タグのみカウント：他の要素が混ざっていても、指定したタグの種類のみをカウントしてスタイルを当てる
    - 例：`p:nth-of-type(2) { color: blue; }`
- `:hover`：要素にマウスカーソルが乗った（ホバーした）状態を指定

    - ホバー時のスタイル変化：ボタンやリンク上にカーソルが移動した際の挙動を指定する
        - 例：`a:hover { opacity: 0.8; }`
- `:not()`：カッコ内に指定した条件「以外」の要素を指定
    - 特定条件の除外：指定した要素やクラスを除外してスタイルを当てる
        - 例：`.list-item:not(:first-child) { margin-top: 16px; }`
- `:focus`：フォーム入力欄などにフォーカス（選択）が当たった状態を指定
    - フォーカス時の指定：入力フォームが選択状態になった際の枠線などを指定する
        - 例：`input:focus { border-color: #007bff; }`
- `:active`：要素がクリックされている（押されている）瞬間の状態を指定
    - クリック時の指定：ボタンを押している最中の見ためを指定する
        - 例：`button:active { background-color: #333; }`
- `:visited`：訪問済み（過去にクリックした）リンクの状態を指定
    - 訪問済みリンクの指定：アクセス履歴のあるリンクの色などを指定する
        - 例：`a:visited { color: #800080; }`

よく使うCSSプロパティ

- `color`：テキストの文字色を指定
    - キーワードによる指定：「black」「white」「red」「blue」など、色名を直接指定する
        - 例：`color: red;`
    - 16進数（HEX）による指定：`#` に続けて16進数で指定する
        - 例：`color: #ff0000;`
    - RGB / RGBA値による指定：赤・緑・青の数値や透明度で指定する
        - 例：`color: rgb(255, 0, 0);`
        - 例：`color: rgba(255, 0, 0, 0.5);`
- `font-size`：テキストの大きさを指定
    - px（ピクセル）による指定：固定サイズで指定する
        - 例：`font-size: 16px;`
    - rem / em（相対値）による指定：ルートや親要素のサイズを基準とした割合で指定する
        - 例：`font-size: 1.5rem;`
    - %（パーセント）による指定：親要素の文字サイズに対する割合で指定する
        - 例：`font-size: 100%;`
- `font-family`：テキストのフォント（書体）を指定
    - フォント名の直接指定：使用するフォント名を指定する（スペースを含む場合は引用符で囲む）
        - 例：`font-family: "Helvetica Neue", Arial, sans-serif;`
    - 総称フォント名による指定：ブラウザ標準の汎用フォントファミリーを指定する
        - 例：`font-family: sans-serif;`
- `font-weight`：テキストの太さを指定

    - キーワードによる指定：標準（`normal`）や太字（`bold`）で指定する
        - 例：`font-weight: bold;`
    - 数値による指定：100〜900の数値（100単位）で指定する
        - 例：`font-weight: 700;`
- `line-height`：行の高さ（行間）を指定
    - 単位なしの数値による指定：文字サイズに対する倍率で指定する（推奨）
        - 例：`line-height: 1.5;`
    - pxによる指定：固定の高さで指定する
        - 例：`line-height: 24px;`
- `text-align`：テキストやインライン要素の水平方向の揃え位置を指定
    - `left` / `start`：左揃え（初期値）
        - 例：`text-align: left;`
    - `center`：中央揃え
        - 例：`text-align: center;`
    - `right` / `end`：右揃え
        - 例：`text-align: right;`
- `width`：要素の横幅を指定
    - pxによる指定：固定の横幅を指定する
        - 例：`width: 300px;`
    - %による指定：親要素の幅に対する割合で指定する
        - 例：`width: 50%;`
    - vwによる指定：画面の横幅に対する割合で指定する
        - 例：`width: 100vw;`
- `height`：要素の高さ（縦幅）を指定
    - pxによる指定：固定の高さを指定する
        - 例：`height: 200px;`
    - %による指定：親要素の高さに対する割合で指定する
        - 例：`height: 100%;`
    - vhによる指定：画面の高さに対する割合で指定する
        - 例：`height: 100vh;`
- `padding`：要素の「内側」の余白を指定
    - 一括指定（ショートハンド）：1〜4つの値で上下左右の内側余白を指定する
        - 例：`padding: 10px 20px;`
    - 個別プロパティによる指定：`padding-top` や `padding-left` など方向を個別で指定する
        - 例：`padding-top: 15px;`
- `margin`：要素の「外側」の余白を指定
    - 一括指定（ショートハンド）：1〜4つの値で上下左右の外側余白を指定する
        - 例：`margin: 0 auto;`
    - 個別プロパティによる指定：`margin-bottom` や `margin-right` など方向を個別で指定する

        - 例：`margin-bottom: 20px;`
- `border`：要素の枠線（太さ・種類・色）を一括指定
    - 一括指定（ショートハンド）:「太さ 種類 色」の順で指定する
        - 例：`border: 1px solid #ccc;`
    - 個別プロパティによる指定：`border-width`、`border-style`、`border-color` を個別指定する
        - 例：`border-style: solid;`
- `border-radius`：要素の角を丸める半径を指定
    - pxによる指定：角丸の半径を数値で指定する
        - 例：`border-radius: 10px;`
    - %による指定：要素サイズに対する割合で指定する（正方形に50%で正円）
        - 例：`border-radius: 50%;`
- `overflow`：要素からはみ出たコンテンツの表示方法を指定
    - `visible`：はみ出た部分をそのまま表示する（初期値）
        - 例：`overflow: visible;`
    - `hidden`：はみ出た部分を非表示（切り取り）にする
        - 例：`overflow: hidden;`
    - `scroll`：スクロールバーを表示して閲覧可能にする
        - 例：`overflow: scroll;`
    - `auto`：必要に応じて自動でスクロールバーを表示する
        - 例：`overflow: auto;`
- `box-shadow`：要素に影を付与する
    - 基本指定：「横方向の距離 縦方向の距離 ぼかしの大きさ 影の色」を指定する
        - 例：`box-shadow: 2px 4px 10px rgba(0, 0, 0, 0.2);`
- `background`：背景に関するスタイル（色・画像・位置など）を一括指定
    - 背景色の指定（`background-color`）：背景の色を指定する
        - 例：`background-color: #f0f0f0;`
    - 背景画像の指定（`background-image`）：背景に表示する画像のパスを指定する
        - 例：`background-image: url('image.jpg');`
    - 一括指定（ショートハンド）：色・画像・繰り返し・位置・サイズなどを1行でまとめて指定する
        - 例：`background: #fff url('bg.png') no-repeat center / cover;`

HTMLの役割

- Webページの**構造**を定義する
- Webサイトは見出し・文章・リンク・画像などの「要素」でできている。HTMLタグは、その要素が何を表しているかの意味を与える
- つまり、コンピューターにWebサイトの構成要素を教えるために使う
- 構造を正しく伝える理由の1つが**アクセシビリティ**

    - 例：目の見えない人が音声読み上げソフトを使うとき、どこが見出しで、どこが文章で、画像に何が写っているかを正しく読み取れる
- 👣 ウェブアクセシビリティ導入ガイドブック（デジタル庁）（卒業後に学ぶ内容）

HTMLを書く順番

次の順で書くとスムーズに書ける。CSSも同じ順で書くとよい。 - 外側 → 内側 - 上 → 下 - 左 → 右

CSSでは、スペルミスや`.`（ドット）漏れ、`-`（ハイフン）と`_`（アンダーバー）の誤用に注意する。

よく使うタグのポイント

タグ	ポイント
`h1`	ページで一番重要な箇所（ページタイトルやヘッダーロゴ）に使う。初学者のうちは「1ページに1つ」と覚えておけばよい
`article`	記事を囲う。似たタグに`section`がある
`time`	時間を表す数字や単語に使う（記事の投稿日時など）
`small`	著作権情報（コピーライト）を書くときに慣習的に使う

- コピーライトの「©」は特殊文字`©`で出せる。他の特殊文字は「特殊文字 HTML」で調べる
- 👣 sectionとarticleの違い／timeタグ（MDN）／HTMLタグ一覧

画像の扱い方

画像の書き出し形式

形式	特徴	用途
jpg	たくさんの色を軽量で表現するのが得意。透過はできない	フルカラーの写真
png	透過ができる。色数が多いとファイルサイズが大きくなる	透過のある写真、色の少ない画像
svg	拡大しても荒くならない。色数が多いと重くなる	ロゴ、アイコン

- 背景透過がいらない画像はjpgで書き出す
- webp・avifなど次世代の画像形式もある。実務では「HTML 画像最適化」などで調べて、その時代に合った形式を使う（👣 次世代画像形式のWebP、AVIF）
- 👣 Figmaから画像を書き出す方法

画像の名前の付け方

最初は分かりやすさ重視でよい。次の点に気をつける。 - 役割や使われている場所が分かる名前（例：`icon_email.svg`、`img_kv_sm.jpg`、`thumb_article.jpg`、`bnr_free-lance.jpg`） - 名前順に並べたときに見やすい名前 - 半角英数字を使う（全角文字やスペースは使わない） - 🖇 よく使っている画像の命名規則（Qiita）／画像ファイルの命名規則（WebNAUT）

imgタグのポイント

- `img`はブロック要素（`div`など）で囲う。CSSで調整しやすくなるため
- `img`タグには`width`・`height`属性をつける。**レイアウトシフト**（画像の読み込みで表示がずれること）を防ぐため
    - 値は画像そのものの大きさでよい（VSCodeで確認できる）
- `alt`属性は実務で非常に重要だが、見落とされがち

```
<div class="sample_img-wrapper">
  <!-- width・heightは画像の元の大きさ。altには画像の内容を書く -->
  <img src="img/hoge.jpg" width="688" height="516" alt="記事のサムネイル" />
</div>
```

- 🖇 imgタグのwidth・heightとレイアウトシフト／alt属性の良い事例／imgタグをpタグなどで囲む理由

レイアウトのポイント

display: flex; をあてる場所

- 「横並びにしたい要素」ではなく、**その親要素**にあてる
- 横並びがうまくいかないときは、あてる場所が正しいか確認する

テキストが折り返されないようにする

- `display: flex;`には「何がなんでも横並びにして、はみ出しそうならサイズを縮めて収める」性質がある。そのため、要素が縮んでテキストが折り返されることがある
- そのときは、折り返しを防ぎたい要素に`white-space: nowrap;`をあてる
- コードは覚えなくてよい。「CSSでテキストの折り返しを防げる」と知っておき、必要なときに「css テキスト 折り返さない」で調べる

2つ目以降だけに余白をつける

```
/* 1つ目以外の要素に、左の余白をつける */
.hoge:not(:first-child) {
  margin-left: 40px;
}
```

- 1つ目の余白は親の`padding`などですでに作れていることが多い
- 👣 CSSの擬似クラスを使いこなそう

リンクのクリック範囲を考える

- ナビゲーション：リンクの高さをナビゲーションいっぱいまで広げると、クリックしやすくなる

```
.sample_nav_link {
  height: 100%;        /* 親の高さいっぱいに広げる（固定値でもよい） */
  display: flex;
  align-items: center; /* 文字は上下中央に置く */
}
```

- ロゴ：左右のクリック範囲を狭めて、誤クリックを防ぐ（`display: flex;`＋`justify-content: center;`、または`inline-block`＋`text-align: center;`など）

画面の高さからヘッダー分を引く

```
.top_kv {
  /* 画面の高さから、ヘッダーの高さ（72px + 72px）を引く */
  height: calc(100vh - 144px);
}
```

- どのデバイスで見ても同じレイアウトになる
- `vh`（ビューハイト）は画面の高さを表す値。`100vh`は画面いっぱい、`50vh`は半分
- 👣 calc()関数の使い方／CSSのvwとvh

PCデザインも意識してコーディングする

```
.top_kv_copy-wrapper {
  width: 100%;      /* 画面が狭いときは画面幅いっぱい */
  max-width: 672px; /* 672pxに達したらそれ以上大きくならない */
}
```

- SPで画面幅いっぱい、PCで672pxというデザインは「画面が広がって672pxに達したら止まる」という仕様を表している
- 初学者のうちは難しいので、模範解答を見て少しずつ慣れる

backgroundのショートハンド

```
/* ショートハンドの書き方 */
background: url("../img/hoge.jpg") center / cover;

/* 分割した書き方（上と同じ意味） */
background-image: url("../img/hoge.jpg");
background-position: center;
background-size: cover;
```

相対パスで画像を読み込む

- 相対パス：コードを書いているファイルをスタート地点にして、対象のファイルまでの道のりを示す書き方
- `../img/hoge.jpg`（style.cssから見た場合）
  - `../`：style.cssのある場所から1つ上の階層に行く（cssフォルダを出る）
  - `img/`：imgフォルダに入る
  - `hoge.jpg`：hoge.jpgを読み込む
- 相対パスと絶対パスの違いと書き方

スクロールバーを消す

```
.hoge {
  overflow: scroll;
  scrollbar-width: none; /* Firefoxなど */
}

.hoge::-webkit-scrollbar {
  display: none; /* Chrome・Safariなど */
}
```

- 暗記は不要。「CSSでスクロールバーを消せる」と知っておけばよい

知っておくと便利なこと

- **hoge**：意味のない名前であることを示す言葉（「山田太郎さん」のような例示用の名前）。`fuga`・`piyo`などもある
- **ベンダープレフィックス**：`-webkit-`・`-moz-`などの文字列。実務では納品前に付けるが、ツールで自動付与もできるので、初学者のうちは気にしなくてよい（☝🏻 ベンダープレフィックスとは）
- **コードの複製**：行を下に複製するショートカット（Mac：`option + shift + ↓`／Windows：`Alt + shift + ↓`）
- **困ったときの進め方**
    1. 何も見ずに書けるか試す（検証ツールで確認しながら）
    2. 授業資料を見直す
    3. 「言語名 やりたいこと」で検索する（例：「css 横並び」）
    4. 復習課題のポイント資料を確認する
    5. 模範解答をちらっと見る
    6. 模範解答を見ながら写経する（1行ずつ役割を意識する）
- 模範解答と全く同じでなくても、ポイントが押さえられていればよい。VSCodeの比較機能で見比べると便利